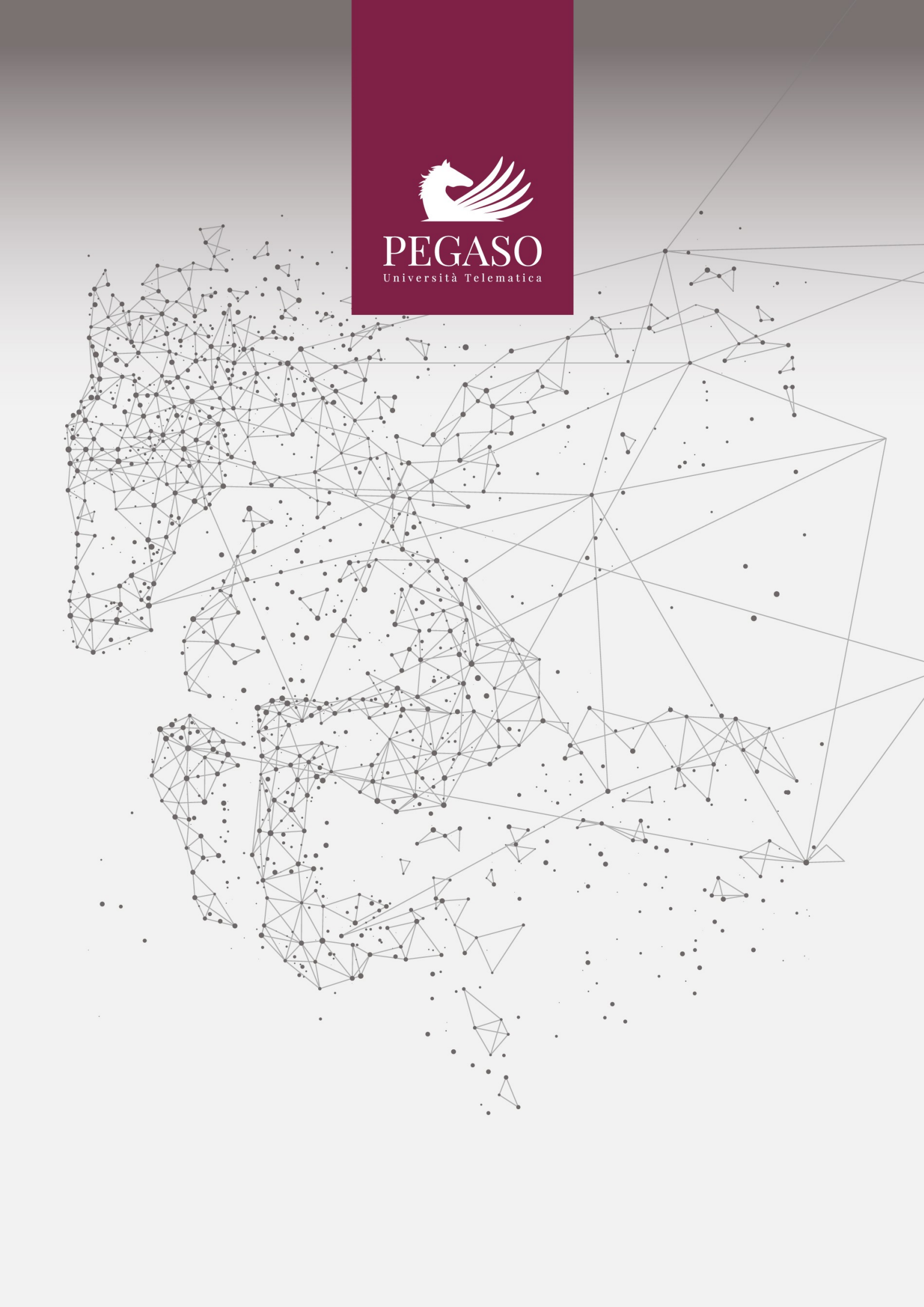

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. ANALISI DEL PROBLEMA 4

3. PROBLEM SOLVING 6

3.1 FUNZIONE AGGIUNGI(NOMEFILE,DESCR,QUANT) 6
3.2 FUNZIONE VISUALIZZA(NOMEFILE) 6
3.3 FUNZIONE RICERCA(NOMEFILE,DESCR) 7
3.4 FUNZIONE MODIFICA(NOMEFILE,DESCR,NEW_QUANT) 7
3.5 FUNZIONE ELIMINA(NOMEFILE,DESCR) 8
3.6 FUNZIONE MAIN() 8

4. FORMALIZZAZIONE DELLA SOLUZIONE 9

4.1 FUNZIONE AGGIUNGI(NOMEFILE,DESCR,QUANT) 9
4.2 FUNZIONE VISUALIZZA(NOMEFILE) 9
4.3 FUNZIONE RICERCA(NOMEFILE,DESCR) 10
4.4 FUNZIONE MODIFICA(NOMEFILE,DESCR,NEW_QUANT) 11
4.5 FUNZIONE ELIMINA(NOMEFILE,DESCR) 12
4.6 FUNZIONE MAIN() 13
4.7 FUNZIONE VISUALIZZA_MENU() 16
4.8 ESEMPIO DI FUNZIONAMENTO DELL'INTERO PROGRAMMA 17
4.9 GESTIONE DELLE ECCEZIONI 18

BIBLIOGRAFIA 25

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta un caso di studio al fine di sviluppare la capacità di programmazione di applicazioni che utilizzano i file per salvare in modo permanente le informazioni in memoria di massa. Viene, inoltre, proposta una strategia di scrittura del codice per la gestione delle eccezioni a partire dal programma già formalizzato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Analisi del problema

Il **caso di studio** che si propone è relativo alla **gestione di un magazzino di una impresa di caffè**[1]. Gli obiettivi di questa lezione sono principalmente due:

1. saper programmare utilizzando i file e gestendo le eccezioni;
2. sviluppare programmi svolgendo un'attività incrementale ed iterativa basata sempre su tre fasi: analisi del problema, problem solving e formalizzazione della soluzione.

La traccia del caso di studio è la seguente:

*"La **Caffè di Mezzanotte** è una piccola società che importa chicchi di caffè crudo da tutto il mondo e li tosta per creare una varietà di miscele. Giulia, proprietaria dell'azienda, ha chiesto di scrivere un programma da utilizzare per la gestione del magazzino. Dopo aver parlato con Giulia, reputiamo necessario utilizzare un file per memorizzare tutti i record del magazzino".*

Ogni record (nel nostro caso uno per caffè) del file dovrebbe avere due campi per contenere questi dati:

1. **descrizione:** una stringa contenente il nome del caffè;
2. **quantità in magazzino:** il numero (in virgola mobile) di chilogrammi presenti in magazzino relativi al nome del caffè.

Una volta identificate le informazioni da salvare sul file per ciascun caffè, è necessario pensare alle attività che Giulia può svolgere sul file secondo l'**approccio top-down** di analisi del problema. Le attività sui file per la gestione del magazzino saranno:

1. aggiunta di tipologie di caffè e relativa quantità in magazzino;
2. visualizzazione del magazzino;
3. ricerca di un caffè nel magazzino;
4. modifica dei dati relativi ad un caffè nel magazzino;
5. eliminazione dei dati relativi ad un caffè nel magazzino.

[1] Esempio tratto dal libro: Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson. La soluzione fornita in questa dispensa sarà parzialmente diversa da quella riportata sul libro.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo caso di studio è possibile far corrispondere le attività sui file ad altrettante funzioni:

1. aggiunta di tipologie di caffè e relativa quantità in magazzino – funzione **aggiungi**();
2. visualizzazione del magazzino – funzione **visualizza**();
3. ricerca di un caffè nel magazzino – funzione **ricerca**();
4. modifica dei dati relativi ad un caffè nel magazzino – funzione **modifica**();
5. eliminazione dei dati relativi ad un caffè nel magazzino – funzione **elimina**().

Per gestire la chiamata delle funzioni sulla base di quanto scelto dall'utente, si utilizzerà un **menu**. Si ricorda che un menu è un **elenco di opzioni,** tra le quali l'utente può scegliere per effettuare un'operazione specifica. La gestione del menu avverrà nella funzione **main**() attraverso una **struttura iterativa e una struttura decisionale (catena di if-elif-else).** Le opzioni offerte dal menu saranno, dunque, 6: 5 opzioni per le attività possibili sul file del magazzino e una per l'uscita dal programma.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Problem solving

A seguito dell'analisi del problema e dell'individuazione delle funzioni e delle opzioni da proporre all'utente del programma (proprietaria dell'azienda) si passa alla **fase di problem solving**.

In questa fase sarà necessario capire come risolvere il problema prima della successiva formalizzazione della soluzione con il coding in Python.

Analizziamo, dunque, per ciascuna delle funzioni individuate nell'analisi del problema:

- input (inteso come parametri passati come argomento);
- output (inteso come valore restituito e/o stampa);
- elaborazione.

3.1 Funzione aggiungi(nomefile,descr,quant)

Input: *nomefile* nome fisico del file, descrizione *descr* del caffè, quantità *quant* del caffè in magazzino.

Output: stampa del messaggio di avvenuta scrittura su file.

Elaborazione: è necessario:

- aprire il file di testo in append;
- scrivere su file la descrizione e la quantità;
- chiudere il file.

3.2 Funzione visualizza(nomefile)

Input: *nomefile* nome fisico del file.

Output: stampa di tutte le descrizioni e le quantità di caffè in magazzino.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Elaborazione: è necessario:

- aprire il file di testo in lettura;
- iterare per leggere da file la descrizione e la quantità corrispondente;
- chiudere il file.

3.3 Funzione ricerca(nomefile,descr)

Input: *nomefile* nome fisico del file, descrizione *descr* del caffè.

Output: stampa della quantità presente in magazzino del caffè individuato dalla descrizione *descr*.

Elaborazione: è necessario:

- aprire il file di testo in lettura;
- iterare per leggere da file e trovare la corrispondenza con la descrizione;
- stampare la quantità (in caso di corrispondenza);
- chiudere il file.

3.4 Funzione modifica(nomefile,descr,new_quant)

Input: *nomefile* nome fisico del file, descrizione *descr* del caffè, nuova quantità *new_quant* del caffè in magazzino (per la modifica).

Output: stampa del messaggio di avvenuta modifica.

Elaborazione: è necessario:

1. aprire il file di testo in lettura;
2. creare un file temporaneo (temp) per scrivere successivamente le righe non modificate;
3. iterare per leggere da file e trovare la corrispondenza con la descrizione (con copia nel file temporaneo);
4. modificare la quantità scrivendola sul file temporaneo (in caso di corrispondenza);

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. terminare iterazione di lettura e copia su file temp;
6. chiudere entrambi i file;
7. eliminare il file originale e ridenominare il file temporaneo con il nome del file originale.

3.5 Funzione elimina(nomefile,descr)

Input: *nomefile* nome fisico del file, descrizione *descr* del caffè.

Output: stampa del messaggio di avvenuta eliminazione.

Elaborazione: è necessario:

1. aprire il file di testo in lettura;
2. creare un file temporaneo (temp) per scrivere successivamente le righe non eliminate;
3. iterare per leggere da file e trovare la corrispondenza con la descrizione (con copia nel file temporaneo);
4. in caso di corrispondenza il record del caffè non viene copiato nel file temporaneo;
5. terminare iterazione di lettura e copia su file temp;
6. chiudere entrambi i file;
7. eliminare il file originale e ridenominare il file temporaneo con il nome del file originale.

3.6 Funzione main()

Input: nessuno.

Output: stampa valori restituiti dalle funzioni.

Elaborazione: è necessario implementare il menu delle opzioni per l'utente e richiamare opportunamente le funzioni viste in precedenza.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formalizzazione della soluzione

Si procede ora a formalizzare il codice sorgente di ciascuna funzione.

4.1 Funzione aggiungi(nomefile,descr,quant)

Alla funzione aggiungi() verranno passati il nome fisico del file (*nomefile*), la descrizione del caffè da aggiungere (*descr*) e la quantità (*quant*). La funzione provvederà ad aprire il file in append e ad aggiungere al file stesso due nuove righe: una contenente la descrizione del caffè e una la quantità corrispondente del caffè. Verrà, quindi, visualizzato a schermo un messaggio di avvenuto salvataggio nel file e, successivamente, verrà chiuso il file. Segue il codice sorgente della funzione:

```
def aggiungi(nomefile,descr,quant):
    outfile = open(nomefile,'a')
    outfile.write(descr+'\n')
    outfile.write(str(quant)+'\n')
    print(descr + " salvato nel file "+nomefile)
    outfile.close()
```

4.2 Funzione visualizza(nomefile)

La **funzione visualizza()** permette di visualizzare a schermo tutte le informazioni relative al magazzino. Alla funzione verrà passato solo il nome fisico del file. La funzione provvederà ad aprire in lettura il file e a richiamare una volta la funzione readline() sull'oggetto file *infile*. Un ciclo while permetterà la lettura del file finché nella variabile *descr* non ci sarà una stringa vuota (segnalazione di end of file). Si ricorda che le righe del file di testo saranno sempre pari e con una numerosità uguale al doppio del numero di caffè in magazzino. Infatti, le righe di indice dispari conterranno la descrizione del caffè e quelle pari la quantità corrispondente.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si provvederà, quindi, ad inserire nella variabile *quant* la quantità di caffè letta dal file e a richiamare il metodo rstrip sulla stringa *descr*. Si visualizzerà l'informazione completa relativa al caffè (descrizione e quantità) e si procederà con la lettura delle righe successive. Infine, si chiuderà il file. Segue il codice sorgente della funzione:

```
def visualizza(nomefile):
    infile = open(nomefile,'r')
    descr = infile.readline()
    while descr!= '':
        quant = float(infile.readline())
        descr = descr.rstrip('\n')

        print("Descrizione: "+descr+" - quantità: "+str(quant)+" kg")
        descr = infile.readline()
    infile.close()
```

4.3 Funzione ricerca(nomefile,descr)

La funzione **ricerca**() utilizza un flag per verificare se il caffè da ricercare è stato trovato o meno. L'inizializzazione del flag è a falso. Alla funzione ricerca() vengono passati il nome fisico del file (*nomefile*) e la descrizione del caffè (*descr*).

La funzione aprirà il file relativo al magazzino del caffè in lettura e, sulla falsariga del codice della funzione visualizza() precedentemente analizzata, procederà alla lettura delle righe del file.

La differenza principale con la funzione visualizza() è l'utilizzo di una struttura decisionale che verifica se la descrizione del caffè letta dal file è la stessa che stiamo cercando. In caso positivo, viene visualizzato a schermo un messaggio in cui si riporta anche la quantità del caffè cercato in magazzino e si imposta il flag a vero. Il flag viene utilizzato principalmente per due ragioni:

1. poterlo eventualmente utilizzare per visualizzare a schermo un messaggio di non presenza del determinato caffè nel file;
2. terminare in anticipo la lettura del file nel caso in cui il caffè da cercare sia stato trovato.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Segue il codice sorgente della funzione:

```
def ricerca(nomefile,descr):
    #uso flag trovato
    trovato = False

    infile = open(nomefile,'r')

    d = infile.readline()
    while d!= '' and not trovato:
        quant = float(infile.readline())
        d = d.rstrip('\n')

        if descr == d:
            print("Caffè trovato - Descrizione: "+descr+" - quantità: "+str(quant)+" kg")
            trovato = True

        d = infile.readline()

    infile.close()

    if not trovato:
        print(descr+" non trovato nel file")
```

4.4 Funzione modifica(nomefile,descr,new_quant)

Alla funzione **modifica**() vengono passati il nome fisico del file (*nomefile*), la descrizione del caffè (*descr*) e la nuova quantità di caffè per la modifica (*new_quant*). La funzione modifica() opera in modo similare alla funzione ricerca(). Le differenze principali con tale funzione sono le seguenti:

- le righe lette che vengono lette dal file *nomefile* vengono copiate in un file temporaneo "*temp.txt*" aperto in scrittura;
- se viene trovata la descrizione *descr* del caffè, nel file temporaneo verranno scritti *descr* e la nuova quantità *new_quant*;
- si prosegue finché non viene analizzato tutto il file originale nella sua interezza.

Al termine vengono chiusi entrambi i file e attraverso le funzioni remove() e rename() disponibili importando il modulo os (operating system – sistema operativo):

- viene eliminato il file originale *nomefile* (remove);
- ridenominato il file temporaneo usando *nomefile* (rename).

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Segue il codice sorgente della funzione:

```
def modifica(nomefile,descr,new_quant):
    #uso flag trovato
    trovato = False

    infile = open(nomefile,'r')
    tempfile = open('temp.txt','w')

    d = infile.readline()
    while d!= '':
        quant = float(infile.readline())
        d = d.rstrip('\n')

        if descr == d:
            tempfile.write(d+'\n')
            tempfile.write(str(new_quant)+'\n')
            trovato = True
        else:
            tempfile.write(d+'\n')
            tempfile.write(str(quant)+'\n')

        d = infile.readline()

    infile.close()
    tempfile.close()

    os.remove(nomefile)
    os.rename('temp.txt',nomefile)

    if trovato:
        print(descr+" aggiornato nel file.")
    else:
        print(descr+" non trovato nel file. File non aggiornato")
```

4.5 Funzione elimina(nomefile,descr)

Alla funzione **elimina**() vengono passati il nome fisico del file (*nomefile*) e la descrizione del caffè (*descr*) da eliminare. La funzione elimina() opera nello stesso modo rispetto alla funzione ricerca(). L'unica differenza è che, se *descr* viene trovato nel file originale, le due righe nel file corrispondenti al caffè da eliminare relative rispettivamente alla descrizione e alla quantità non vengono copiate nel file temporaneo.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Segue il codice sorgente della funzione:

```
def elimina(nomefile,descr):
    #uso flag trovato
    trovato = False

    infile = open(nomefile,'r')
    tempfile = open('temp.txt','w')

    d = infile.readline()
    while d!= '':
        quant = float(infile.readline())
        d = d.rstrip('\n')

        if descr != d:
            tempfile.write(d+'\n')
            tempfile.write(str(quant)+'\n')

        else:
            trovato = True

        d = infile.readline()

    infile.close()
    tempfile.close()

    os.remove(nomefile)
    os.rename('temp.txt',nomefile)

    if trovato:
        print(descr+" eliminato dal file. File aggiornato")
    else:
        print(descr+" non trovato nel file. File non aggiornato")
```

4.6 Funzione main()

Le opzioni individuate come nome e numero diventano le seguenti costanti globali che vengono dichiarate al di fuori della funzione main():

```
ADD = 1
VIEW = 2
SEARCH = 3
MODIFY = 4
DELETE =5
QUIT = 6
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per evitare di appesantire il codice della funzione main() con una serie di chiamate alla funzione print() per mostrare tutte le opzioni possibili del menu è consigliabile utilizzare un'altra funzione, la funzione **visualizza_menu**() (vedi paragrafo 4.7).

Ritornando alla funzione main(), si inizializzerà il nome fisico del file che rappresenta il magazzino del caffè.

Per la gestione del menu, si utilizzerà una variabile numerica intera *scelta* per la selezione dell'opzione dell'utente. La variabile *scelta* verrà inizializzata ad un valore al di fuori di quello logicamente valido (in questo caso 0 fuori dall'intervallo 1-6).

L'inizializzazione precedente permette di entrare in un ciclo while che itera finché la scelta dell'utente è diversa dall'opzione di uscita (nel nostro caso 6). A seguito dell'input da parte dell'utente:

- se la scelta è fuori dall'intervallo delle opzioni (nel nostro caso 1-6), si chiederà all'utente di ripetere la scelta;
- se la scelta è nell'intervallo, una struttura decisionale richiamerà le funzioni corrispondenti in modo opportuno.

Il codice seguente imposta il gestore del menu:

```
def main():

    scelta = 0
    nomefile = "caffè.txt"

    while scelta!=QUIT:
        visualizza_menu()
        scelta = int(input("Selezionare una delle opzioni (1-6):\t"))
        print()
```

La struttura decisionale all'interno del ciclo while, invece, permette di eseguire le richieste dell'utente. In base all'opzione (espressa come numero intero) decisa dall'utente, la funzione provvederà a prendere in input i dati necessari per le attività e a chiamare la funzione corrispondente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per la prima opzione **"Aggiungi caffè"**, vengono richieste la descrizione del caffè (*descr*) e la quantità relativa (*quant*) per, poi, chiamare la funzione **aggiungi**() passando i parametri *nomefile, descr* e *quant*.

Per la seconda opzione **"Visualizza magazzino caffè"**, non viene richiesto nessun input e vengono visualizzate tutte le informazioni del magazzino attraverso la chiamata alla funzione **visualizza**() a cui verrà passato solo *nomefile*.

Per la terza opzione **"Ricerca quantità di un caffè nel magazzino"**, viene richiesta solo la descrizione del caffè (*descr*) da ricercare. Verrà, dunque, chiamata la funzione ricerca() a cui verranno passati *nomefile* e *descr*.

Per la quarta opzione **"Modifica quantità di un caffè nel magazzino"**, vengono richieste la descrizione del caffè (*descr*) e la nuova quantità relativa da aggiornare (*quant*) per, poi, chiamare la funzione **modifica**() passando i parametri *nomefile, descr* e *quant*..

Per la quinta opzione **"Elimina caffè"**, viene richiesta solo la descrizione del caffè (*descr*) da eliminare. Verrà, dunque, chiamata la funzione elimina() a cui verranno passati *nomefile* e *descr*.

Per la sesta ed ultima opzione **"Uscire dal programma"**, non viene richiesto nessun input. Si visualizza a schermo solo un messaggio per comunicare all'utente che il programma verrà terminato.

L'else finale serve per intercettare tutte le scelte al di fuori dell'intervallo [1-6] al fine di comunicare all'utente della scelta non valida e ripetere la scelta dell'opzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Segue il codice sorgente della funzione:

```
if scelta == ADD:
    descr = input("Descrizione caffè: ")
    quant = float(input("Quantità (in kg): "))
    aggiungi(nomefile,descr,quant)
    print()

elif scelta == VIEW:
    visualizza(nomefile)
    print()

elif scelta == SEARCH:
    descr = input("Descrizione caffè da ricercare: ")
    ricerca(nomefile,descr)
    print()

elif scelta == MODIFY:
    descr = input("Descrizione caffè: ")
    quant = float(input("Inserire nuova quantità (in kg): "))
    modifica(nomefile,descr,quant)
    print()

elif scelta == DELETE:
    descr = input("Descrizione caffè: ")
    elimina(nomefile,descr)
    print()

elif scelta == QUIT:
    print("Esco dal programma...")

else:
    print("Errore, scelta non valida")
```

4.7 Funzione visualizza_menu()

La funzione **visualizza_menu**() ha il solo scopo di stampare a schermo le opzioni del menu.

Serve principalmente a ridurre il numero di linee di codice all'interno della funzione main() per aumentarne la leggibilità.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Segue il codice sorgente della funzione:

```
def visualizza_menu():
    print("\n\tMENU")
    print("1. Aggiungi caffè")
    print("2. Visualizza magazzino caffè")
    print("3. Ricerca quantità di un caffè nel magazzino")
    print("4. Modifica quantità di un caffè nel magazzino")
    print("5. Elimina caffè")
    print("6. Uscire dal programma\n")
```

4.8 Esempio di funzionamento dell'intero programma

Di seguito si riporta un esempio di esecuzione del programma. Per esigenze di spazio, si riporta un caso molto semplice di aggiunta di un caffè con la relativa quantità e la successiva stampa delle informazioni del magazzino. Il lettore può sperimentare l'esecuzione del codice con la scelta delle diverse opzioni disponibili.

```
        MENU
1. Aggiungi caffè
2. Visualizza magazzino caffè
3. Ricerca quantità di un caffè nel magazzino
4. Modifica quantità di un caffè nel magazzino
5. Elimina caffè
6. Uscire dal programma

Selezionare una delle opzioni (1-6):    1

Descrizione caffè: arabica
Quantità (in kg): 4.5
arabica salvato nel file caffè.txt


        MENU
1. Aggiungi caffè
2. Visualizza magazzino caffè
3. Ricerca quantità di un caffè nel magazzino
4. Modifica quantità di un caffè nel magazzino
5. Elimina caffè
6. Uscire dal programma

Selezionare una delle opzioni (1-6):    2

Descrizione: arabica - quantità: 4.5 kg
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4.9 Gestione delle eccezioni

Una volta **terminato e verificato il programma** è buona norma inserire opportunamente il codice per la gestione delle eccezioni. Questo permette di evitare l'interruzione brusca del programma e la potenziale perdita di dati.

Si procede individuando tutte quelle istruzioni che possono potenzialmente sollevare eccezioni funzione per funzione.

Nella funzione **aggiungi**():

- l'apertura del file è in append quindi non solleverà eccezioni;
- il metodo write accetta solo stringhe come parametro, quindi nel caso in cui alla funzione vengano passati dei valori di altri tipo (es. numerici), potrebbe essere sollevata un'eccezione;
- tutte le operazioni di concatenazione di stringhe richiedono stringhe. Come nel caso precedente potrebbe essere sollevata un'eccezione.

Il codice della funzione aggiungi() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def aggiungi(nomefile,descr,quant):

    try:
        outfile = open(nomefile,'a')
        outfile.write(descr+'\n')
        outfile.write(str(quant)+'\n')
        print(descr + " salvato nel file "+nomefile)
        outfile.close()

    except ValueError:
        print("Quantità di caffè non numerica")
        infile.close()

    except:
        print("Si è verificato un errore")
        infile.close()
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nella funzione **visualizza**():

- l'apertura del file è in lettura quindi può sollevare eccezioni;
- la funzione di casting a float potrebbe sollevare un'eccezione se la stringa non contenesse un numero;
- tutte le operazioni di concatenazione di stringhe richiedono stringhe.

Il codice della funzione visualizza() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def visualizza(nomefile):
    try:
        infile = open(nomefile,'r')
        descr = infile.readline()
        while descr!= '':
            quant = float(infile.readline())
            descr = descr.rstrip('\n')

            print("Descrizione: "+descr+" - quantità: "+str(quant)+" kg")
            descr = infile.readline()

    except FileNotFoundError:
        print("File non esistente. Controllare il nome del file")

    except ValueError:
        print("Quantità di caffè non numerica")
        infile.close()

    except:
        print("Si è verificato un errore")
        infile.close()

    else:
        infile.close()
```

Nella funzione **ricerca**():

- l'apertura del file è in lettura quindi può sollevare eccezioni;
- la funzione di casting a float potrebbe sollevare un'eccezione se la stringa non contenesse un numero;
- tutte le operazioni di concatenazione di stringhe richiedono stringhe.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice della funzione ricerca() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def ricerca(nomefile,descr):
    #uso flag trovato
    trovato = False

    try:
        infile = open(nomefile,'r')

        d = infile.readline()
        while d!= '' and not trovato:
            quant = float(infile.readline())
            d = d.rstrip('\n')

            if descr == d:
                print("Caffè trovato - Descrizione: "+descr+" - quantità: "+str(quant)+" kg")
                trovato = True

            d = infile.readline()

        if not trovato:
            print(descr+" non trovato nel file")

    except FileNotFoundError:
        print("File non esistente. Controllare il nome del file")

    except ValueError:
        print("Quantità di caffè non numerica")
        infile.close()

    except:
        print("Si è verificato un errore")
        infile.close()

    else:
        infile.close()
```

Nella funzione **modifica**():

- l'apertura del file è in lettura quindi può sollevare eccezioni;
- la funzione di casting a float potrebbe sollevare un'eccezione se la stringa non contenesse un numero;
- tutte le operazioni di concatenazione di stringhe richiedono stringhe.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice della funzione modifica() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def modifica(nomefile,descr,new_quant):
    #uso flag trovato
    trovato = False

    try:
        infile = open(nomefile,'r')
        tempfile = open('temp.txt','w')

        d = infile.readline()
        while d!= '':
            quant = float(infile.readline())
            d = d.rstrip('\n')

            if descr == d:
                tempfile.write(d+'\n')
                tempfile.write(str(new_quant)+'\n')
                trovato = True
            else:
                tempfile.write(d+'\n')
                tempfile.write(str(quant)+'\n')

            d = infile.readline()

        tempfile.close()

        os.remove(nomefile)
        os.rename('temp.txt',nomefile)

        if trovato:
            print(descr+" aggiornato nel file.")
        else:
            print(descr+" non trovato nel file. File non aggiornato")


    except FileNotFoundError:
        print("File non esistente. Controllare il nome del file")

    except ValueError:
        print("Quantità di caffè non numerica")
        infile.close()

    except:
        print("Si è verificato un errore")
        infile.close()

    else:
        infile.close()
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nella funzione **elimina**():

- l'apertura del file è in lettura quindi può sollevare eccezioni;
- la funzione di casting a float potrebbe sollevare un'eccezione se la stringa non contenesse un numero;
- tutte le operazioni di concatenazione di stringhe richiedono stringhe.

Il codice della funzione elimina() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def elimina(nomefile,descr):
    #uso flag trovato
    trovato = False
    try:
        infile = open(nomefile,'r')
        tempfile = open('temp.txt','w')

        d = infile.readline()
        while d!= '':
            quant = float(infile.readline())
            d = d.rstrip('\n')

            if descr != d:
                tempfile.write(d+'\n')
                tempfile.write(str(quant)+'\n')

            else:
                trovato = True

            d = infile.readline()


        tempfile.close()

        os.remove(nomefile)
        os.rename('temp.txt',nomefile)

        if trovato:
            print(descr+" eliminato dal file. File aggiornato")
        else:
            print(descr+" non trovato nel file. File non aggiornato")

    except FileNotFoundError:
        print("File non esistente. Controllare il nome del file")

    except ValueError:
        print("Quantità di caffè non numerica")
        infile.close()

    except:
        print("Si è verificato un errore")
        infile.close()

    else:
        infile.close()
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nella funzione **visualizza_menu**() le chiamate a print per come sono scritte non comportano il rischio di eccezioni. Il codice di questa funzione, pertanto, rimane invariato.

Nella funzione **main**():

- le funzioni di casting a int e a float potrebbero sollevare un'eccezione se la stringa non contenesse un numero.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice della funzione main() integrato del codice per la gestione delle eccezioni diventa, dunque:

```
def main():

    scelta = 0
    nomefile = "caffè.txt"

    while scelta!=QUIT:
        visualizza_menu()

        try:
            scelta = int(input("Selezionare una delle opzioni (1-6):\t"))
        except ValueError:
            print("Inserire un valore numerico intero")
            scelta=0

        print()

        if scelta == ADD:
            try:
                descr = input("Descrizione caffè: ")
                quant = float(input("Quantità (in kg): "))
                aggiungi(nomefile,descr,quant)

            except ValueError:
                print("Inserire una quantità di caffè numerica. Ripetere l'operazione")
            print()

        elif scelta == VIEW:
            visualizza(nomefile)
            print()

        elif scelta == SEARCH:
            descr = input("Descrizione caffè da ricercare: ")
            ricerca(nomefile,descr)
            print()

        elif scelta == MODIFY:
            try:
                descr = input("Descrizione caffè: ")
                quant = float(input("Inserire nuova quantità (in kg): "))
                modifica(nomefile,descr,quant)
            except ValueError:
                print("Inserire una quantità di caffè numerica. Ripetere l'operazione")
            print()

        elif scelta == DELETE:
            descr = input("Descrizione caffè: ")
            elimina(nomefile,descr)
            print()

        elif scelta == QUIT:
            print("Esco dal programma...")

        else:
            print("Errore, scelta non valida")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).